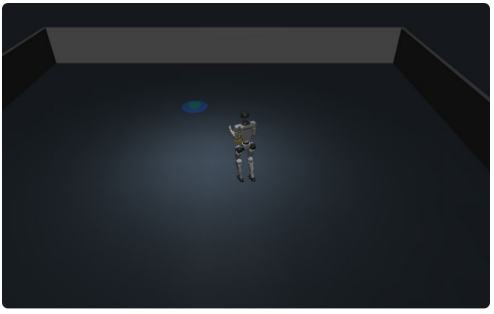
Watching a robot do its job

A monitor that knows the stages of a task

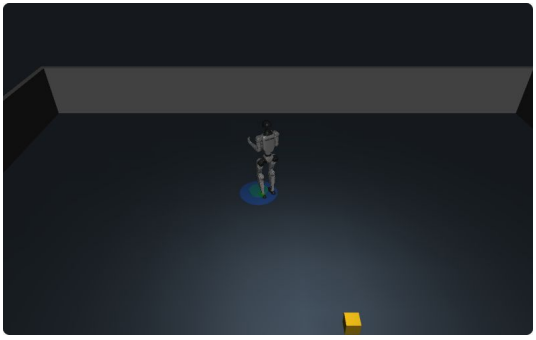
Pinhas Nehoray Aburmad
Supervisor: Prof. Ronen Brafman
Ben-Gurion University of the Negev
Progress meeting, October 2026



Unitree G1 humanoid in the MuJoCo simulator.

The problem: robots fail silently

- A robot does a task step by step: walk, pick up, carry, put down.
- Things go wrong: the box slips, the grasp misses, someone pushes it.
- Often the robot **does not know**. It reports "done" anyway.
- We want an **independent watcher** that notices, **as soon as it happens**.

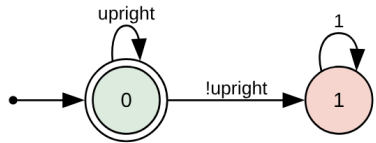


The robot is at the drop-off (blue) and reports success.

The box (yellow) is still lying on the floor behind it.

Background: what is a runtime monitor?

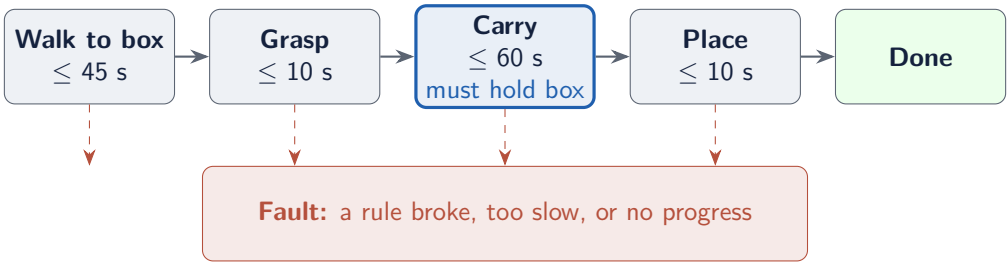
- A **separate program** that only listens to the robot's messages: where it is, what its laser sees, where the box is.
- It checks **rules over time**, written in **LTL** (linear temporal logic): "*always* stay upright", "*eventually* deliver the box".
- A tool (Spot) turns each rule into a small **state machine** (automaton). Every second the monitor takes one step.
- The catch: "eventually" **never fails early**. It can always still happen later.



The rule "always upright" as an automaton: stay in 0 while upright; one fall moves to 1 (red) for good.

Our idea: give the monitor the stages of the task

- Split the task into **stages** ("progress states"), like the steps of a recipe.
- Each stage has its own rules: what must **stay true**, when it **ends**, and a **time limit**.
- Example: *while carrying, the robot must hold the box.* A drop breaks this rule at once.

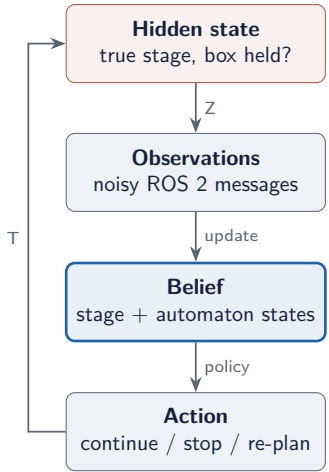


The four stages the monitor generated for "fetch and deliver".

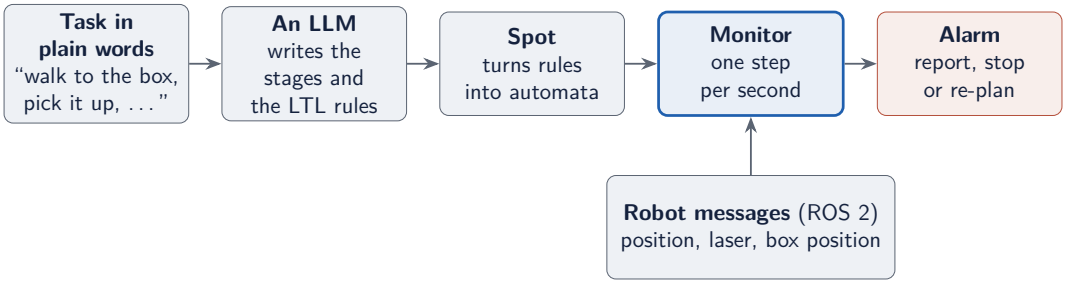
The model behind it: a POMDP

S	hidden: robot, box, *true stage*, fault yes/no
O	ROS 2 messages: position, laser, box position
Z	noise, missed detections; "held" is inferred
T	physics, the executor, unannounced faults
A	continue, warn, stop, re-plan
R	on time, early detection, few false alarms

- **Stages make observations informative:** "not holding" is normal while walking, a fault while carrying.
- **Early knowledge pays only with a deadline** (value of information). We test exactly this in Experiment 2.
- Today: a **deterministic belief** and a fixed rule, not a solver. Next: a probabilistic belief and an optimised policy.



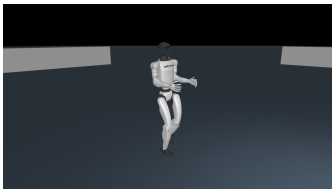
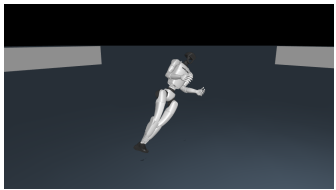
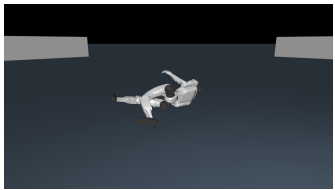
Method: how the monitor is built and run



- The rules are generated from the description **in one try** and checked automatically.
- The monitor is its **own program**. It reads only sensor messages and **never asks the robot's planner** how it is doing.

Test bed: a humanoid in a physics simulator

- Unitree **G1** humanoid in **MuJoCo**, walking with Unitree's own controller.
- We **inject faults on purpose**: push, slip, weak legs, stall, dropped box, failed grasp, ...
- Each run is judged by the simulator's **true outcome**, which the monitor never sees.
- Main comparison: the monitor **with stages** vs. the same rules **without stages** ("flat").

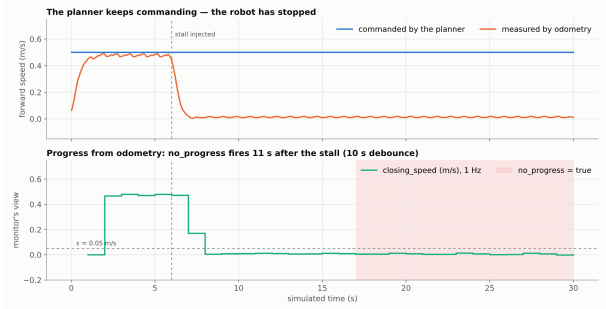
Walking → pushed → on the floor: the monitor flags the fall within 1 second.

Experiment 1: a short walk

1800 runs

- Only two stages, and both check the same thing, so stages **added little**: stalls caught 15 vs. 14 of 15.
- What mattered was measuring **progress from the robot's own sensors**. Without that check: **1 of 15** stalls caught.
- The planner keeps saying "moving" while the robot is stuck.

Lesson: never trust the planner's own report.

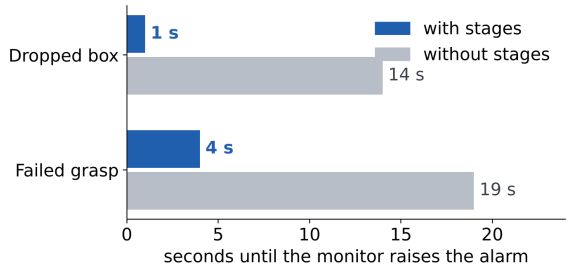


Top: the planner commands 0.5 m/s, the robot has stopped. Bottom: the monitor's own measure of progress drops, and it raises "no progress".

Experiment 2: fetch and deliver

3360 runs, over ROS 2

- Walk to a box, grasp, carry, place: **four stages**.
- Dropped box: alarm after **1 s** with stages, 14 s without.
- Failed grasp: **4 s** vs. 19 s.
- Earlier in **20 of 20** runs ($p = 1.9 \cdot 10^{-6}$).
- Without stages the monitor notices only when the robot claims "done".

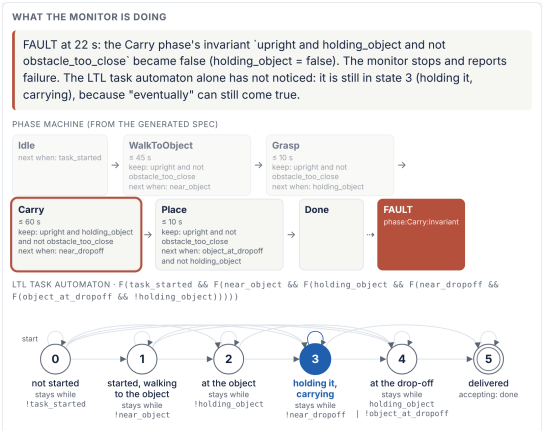


Median over 20 runs of each fault.

Inside one run: the stage catches the drop

- **17 s:** Carry starts. From now on the box must be held.
- **21.6 s:** the box drops.
- **22 s: alarm.** The Carry rule broke.
- The plain LTL automaton is **still in state 3** (“carrying”): “eventually” can still come true.

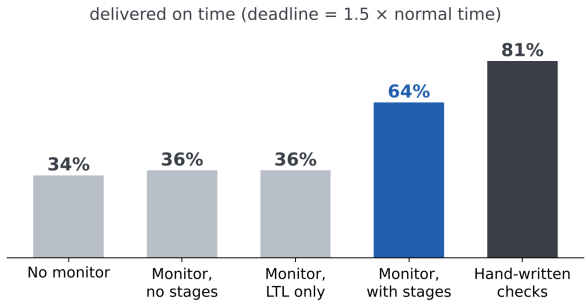
The stage did the catching.



From the live demo (one run): the stages on top, the LTL automaton below; current stage and state lit.

Does catching early matter? Yes, when time matters

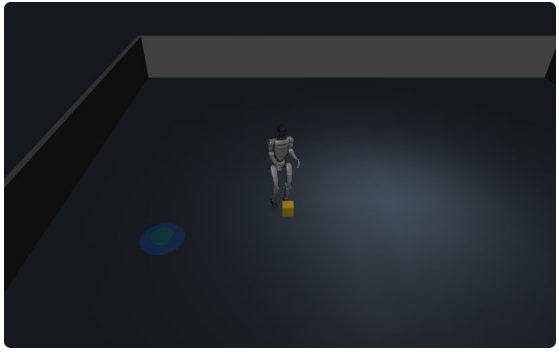
- With no deadline, re-planning fixes things later: every monitor ends at about **82–84%**.
- With a deadline: **64%** delivered with stages vs. 36% without. LTL alone: also 36%.
- Hand-written checks inside the robot program: 81%. But someone had to **foresee each fault and code it**.



160 runs per bar, all fault types.

Faults nobody wrote a check for

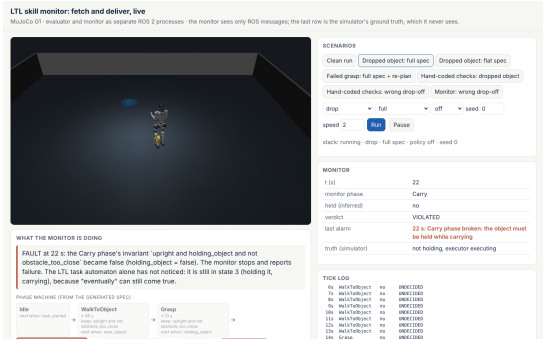
- New faults that the hand-written checks do not cover: the **wrong drop-off place**, and passing **too close** to walls.
- Hand-written checks alone: **40 of 40** failed **silently**.
- Adding our monitor: **0 of 40** silent, at no cost elsewhere.
- Missed by everyone: going slowly and detours that stay inside the time limits.



The robot put the box down far from the drop-off pad (blue) and reports success. The monitor flags it.

What we learned

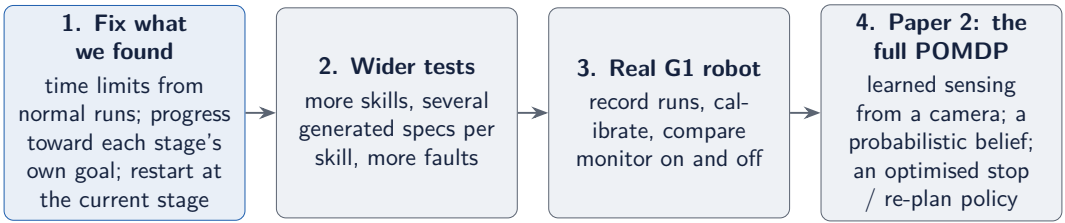
- **Stages catch a failure when it happens**, not at the end.
- Early detection **pays off when time matters**.
- The monitor is **generated from plain words**, needs no task code, and catches faults nobody planned for.
- Progress must come from the **robot's own sensors**, not the planner.



Everything runs live in one command:

`docker run -rm -p 8080:8080 ghcr.io/pinhas019/ltlmonitor-demo`

To be done



- **Honest limits today:** the grasp is simulated (the G1 model has no arms), the box position comes from a simple stand-in for a camera, and thresholds are not yet calibrated.